big data and cognitive computing

MDPI

Article

Improved Particle Swarm Optimization Based on Fuzzy Controller Fusion of Multiple Strategies for Multi-Robot Path Planning

Jialing Hu, Yanqi Zheng, Siwei Wang and Changjun Zhou *

School of Computer Science and Technology, Zhejiang Normal University, Jinhua 321004, China
* Correspondence: zhouchangjun@zjnu.edu.cn

Abstract

Robots play a crucial role in experimental smart cities and are ubiquitous in daily life, especially in complex environments where multiple robots are often needed to solve problems collaboratively. Researchers have found that the swarm intelligence optimization algorithm has a better performance in planning robot paths, but the traditional swarm intelligence algorithm cannot be targeted to solve the robot path planning problem in difficult problem. Therefore, this paper aims to introduce a fuzzy controller, mutation factor, exponential noise, and other strategies on the basis of particle swarm optimization to solve this problem. By judging the moving speed of different particles at different periods of the algorithm, the individual learning factor and social learning factor of the particles are obtained by fuzzy controller, and using the leader particle and random particle, designing a new dynamic balance of mutation factor, with the iterative process of the adaptation value of continuous non-updating counter and continuous updating counter to control the proportion of the elite individuals and random individuals. Finally, using exponential noise to update the matrix of the population every 50 iterations is a way to balance the local search ability and global exploration ability of the algorithm. In order to test the proposed algorithm, the main method of this paper is simulated on simple scenarios, complex scenarios, and random maps consisting of different numbers of static obstacles and dynamic obstacles, and the algorithm proposed in this paper is compared with eight other algorithms. The average path deviation error of the planned paths is smaller; the average distance of untraveled target is shorter; the number of steps of the robot movements is smaller, and the path is shorter, which is superior to the other eight algorithms. This superiority in solving multi-robot cooperative path planning has good practicality in many fields such as logistics and distribution, industrial automation operation, and so on.

Keywords: robots; path planning; particle swarm optimization; mutation factor; fuzzy controller; exponential noise

Academic Editors: Domenico Ursino and Min Chen

Received: 25 June 2025
Revised: 17 August 2025
Accepted: 28 August 2025
Published: 2 September 2025

Citation: Hu, J.; Zheng, Y.; Wang, S.; Zhou, C. Improved Particle Swarm Optimization Based on Fuzzy Controller Fusion of Multiple Strategies for Multi-Robot Path Planning. *Big Data Cogn. Comput.* **2025**, *9*, 229. https://doi.org/10.3390/bdcc9090229

1. Introduction

With the rapid development of robotic technology, the use of robots in various fields is becoming more and more widespread, including autonomous driving, industrial operations, disaster emergency response, logistics and distribution, etc, playing a vital role in advancing the development of smart cities. Ye et al. [1] used a robot for path planning research in narrow passages; Safa et al. [2] used a robot for path planning to improve the robot's ability to complete tasks in industrial environments; Liu et al. [3] implemented

cross-country path planning with robots to determine the rescue plan faster in disaster rescue; Wang et al. [4] used robots for object localization and grasping; Liang et al. [5] used robots in the field of deep-sea mining; Huang et al. [6] used robots for external building detection, and Barnaw et al. [7] used robots in the field of landmine detection. Zhao et al. [8] have realized the use of robots to assist in the establishment of a disaster response network.

Path planning is a crucial aspect in robotic task execution, affecting the efficiency and safety of task execution. A single robot can no longer perform these tasks efficiently, so collaborative planning of multiple robots to complete the task is a very efficient practice. Path planning is a key aspect in robot navigation and control, which determines the forward trajectory of the robot from the starting point to the end point. For a single robot, path planning mainly focuses on how to avoid obstacles and find an optimal path. However, in the scenario of multiple robots working together, path planning needs to consider not only the forward trajectory of each robot, but also the collaboration and obstacle avoidance among multiple robots to ensure that the robots can complete the task safely and efficiently. Zhou et al. [9] proposed a path planning method for autonomous robots oriented to indoor blind zones. Tang et al. [10] proposed a novel discrete multi-objective cross-entropy optimization algorithm to solve the path planning problem of collaborative arc welding with two robots. Huo et al. [11] proposed a new method for smooth path planning of an Ackermann mobile robot based on an improved Ant Colony Optimization (ACO) algorithm, a bio-inspired algorithm simulating ants' foraging behavior to solve optimization problems, and a B-spline curve, a parametric curve defined by control points, basis functions, and a knot vector.

In terms of 2D path planning, mobile robot path planning is often divided into single robot path planning and multiple robot path planning. In terms of single robot path planning, Tao et al. [12] proposed a two-population particle swarm optimization based on a random perturbation strategy, which can guarantee the quality of the particles while fully searching the global map and has a better performance in mobile robot path planning. Cui et al. [13] proposed a multi-strategy adaptive ACO optimization algorithm, which improves the ACO algorithm through a direction guiding mechanism, adaptive heuristics, state transfer probability rules, and a pheromone initialization method, in solving the problem of multiple-robot path planning function, state transfer probability rule, and a pheromone initialization method to improve the ACO algorithm, which is able to generate shorter and smoother paths in solving individual robot path planning. Li et al. [14] proposed an improved ACO algorithm, which is superior in solving the problem of static obstacle avoidance and dynamic obstacle avoidance by mobile robots; Rustu et al. [15] proposed an adaptive differential sine cosine algorithm with a policy pooling approach, which was applied to solve the multi-robot path planning problem. Mustafa et al. [16] proposed an improved artificial bee colony (IABC) algorithm for grid-based UAV path planning. Integrating linearisation and local search strategies, it is suitable for autonomous navigation tasks in complex obstacle environments, demonstrating robust performance and adaptability to diverse settings. Tian et al. [17] designed the jumping particle swarm optimization capable of safe obstacle avoidance, which can better solve the dynamic obstacle avoidance problem among multiple robots. Mac et al. [18] proposed a multi-objective particle swarm optimization algorithm in order to solve the path planning problem of mobile robots in complex chaotic problems, in order to achieve shorter path lengths and smoother paths. Das et al. [19] designed an improved particle swarm optimization, which can generate paths faster, and the generated paths are safer and more energy-efficient. Bandita [20] implemented path planning for robots by mixing the improved cuckoo algorithm with particle swarm optimization. Bandita [21] also designed an improved particle swarm opti-

mization that introduced Q-Learning and a distributed democratic approach; the proposed algorithm is superior in terms of synchronization, collaboration, and path traveling.

Swarm intelligence algorithms are a class of optimization algorithms inspired by the behavior of groups in nature, which solve complex problems or find optimal solutions by simulating the collaboration and interaction between individuals in a group. In this field, classical algorithms, such as particle swarm optimization [22] and grey wolf algorithm [23], have been widely used. And in recent years, new swarm intelligence algorithms have emerged, which demonstrate unique optimization performance by simulating different biological or social behaviors. For example, the whale optimization algorithm proposed by Mirjalili [24] is one of the best. It achieves an efficient optimization process by simulating the hunting behavior of a humpback whale group in three phases, including searching for food, shrinking encirclement, and spiral updating of position. This algorithm has attracted much attention for its concise control parameters and independent population updating mechanism. Xue et al. [25], on the other hand, proposed the sparrow search algorithm, which was modeled on the foraging behavior of sparrow flocks and demonstrated a strong global search capability and the ability to quickly jump out of the local optimum. Zhu [26] proposed the human memory optimization algorithm, which is also an innovative study. This algorithm draws on the human memory mechanism and gradually approaches the optimal solution of the problem by simulating the human memory mode and behavior. These emerging group intelligence algorithms not only enrich the theoretical system of optimization algorithms but also provide new ideas and methods for solving complex problems. Yang et al. [27] proposed a firefly optimization algorithm, which simulates the flickering behavior of fireflies, in which the fireflies are able to attract the movement of other fireflies by adjusting their own brightness to achieve the optimization. The algorithm has fewer parameters that are easy to be adjusted, and it has a strong global search ability and has better adaptability in some problems. Yang et al. [28] also proposed the cuckoo algorithm by imitating the cuckoo bird nest searching and egg-laying activities, which received inspiration from the parasitic behavior of cuckoos. The cuckoo search algorithm achieves the balance of global and local search through Lévy flights, which has better convergence and randomness, and helps to find the optimal solution quickly while maintaining population diversity. There are also various excellent swarm intelligence optimization algorithms, such as Artificial Bee Colony Algorithm [29], Drosophila Optimisation Algorithm [30], Dragonfly Optimisation Algorithm [31], Bat Algorithm [32], Fish Swarm Optimisation Algorithm [33], and so on.

However, the traditional swarm intelligence algorithm still exists problems, such as a tendency to fall into the local optimal solution, and the stability and adaptive ability of the algorithm are insufficient, so many experts and scholars have improved the traditional optimization algorithms proposed by their predecessors and put forward a lot of improved swarm intelligence algorithms with excellent performance. For example, Li et al. [34] improved the Whale Optimization Algorithm (WOA), a bio-inspired optimization algorithm that mimics the hunting behavior of humpback whales, and they made WOA strengthen its global and local search ability while avoiding local optimal solutions by introducing innovative means such as a priori parameter intervals, variable enclosing factors, nonlinear convergence factors, etc. Yu et al. [35] proposed an improved Whale Optimization Algorithm, which adopts a new population initialization method and introduces escape energy and wormhole search strategies, aiming to improve the algorithm's local search capability. Zhu et al. [36], on the other hand, designed a manta ray foraging algorithm for a multi-optimization problem, which combines a matching game, asymptotic learning, and an optimal position updating strategy in order to improve the algorithm's performance when dealing with complex multi-objective problems. Wu et al. [37] proposed a multi-ant

colony optimization algorithm based on a cooperative game and dynamic path tracking. They improved the convergence of the algorithm by constructing a novel heterogeneous multinomial ant colony and introducing a cooperative game and dynamic path tracking mechanism. Wang et al. [38] developed an improved grey wolf optimization algorithm (IGWO), which integrates evolutionary and elimination mechanisms to effectively improve the performance and adaptability of the algorithm. Luo et al. [39], on the other hand, constructed a more realistic model that can reflect the leadership hierarchy of grey wolves in nature, which provides more realistic theoretical support for the grey wolf optimization algorithm. Dhargupta et al. [40] proposed an innovative SOGWO algorithm, which cleverly combines the binary learning strategy with the grey wolf optimization algorithm, making fuller use of the learning space.

Traditional path planning methods, such as the artificial potential field method [41], Dijkstra's algorithm [42], A* algorithm [43], RRT algorithm [44], etc., although they can satisfy the path planning needs of individual robots to a certain extent, they often suffer from the problems of large computation and low efficiency when dealing with the collaborative path planning of multiple robots. Therefore, this thesis aims to propose an improved particle swarm optimization for collaborative path planning of multiple robots. By introducing the improved algorithm, we aim to solve the static obstacle and dynamic obstacle avoidance problems in multiple robots' path planning and improve the efficiency and feasibility of path planning. Validated by the above research work, swarm intelligence algorithms show a strong global search capability, and these algorithms possess a high degree of flexibility and adaptability, which can be improved according to different practical problems and needs, thus achieving satisfactory results in various application scenarios. Among them, particle swarm optimization (PSO), as an optimization algorithm based on group intelligence, has been widely used in the field of path planning due to its advantages of simple implementation and fast convergence speed.

In this paper, the improved particle swarm optimization is applied to the study of robot path planning, and the innovations and work of this paper are as follows:

(1) For the two learning factors in the particle swarm optimization, this paper combines the movement speed of particles and the current iteration number to design a fuzzy controller;
(2) Aiming at the situation that the particle swarm is easy to fall into the local optimal solution, this paper designs a dynamic balanced mutation factor and uses a counter to count the continuous update and continuous non-update of the particle adaptation value to adjust the proportion of the leader particle among the mutated particles;
(3) After every 50 iterations, exponential noise is used to update the population matrix;
(4) Simulation experiments were carried out in simple, complex, and stochastic models.

This paper is structured as follows: in Section 2, the online path planning model is described; in Section 3, the traditional particle swarm optimization, the motivation for improvement, and the proposed algorithm are described; in Section 4, the simulation experiments are analyzed; and in Section 5, this paper is concluded, and future efforts are indicated.

2. Online Path Planning Models

Path planning for mobile robots requires planning a path that has the shortest path length, the smallest difference from the ideal path, and the lowest number of steps to move. In this paper, the mobile robots are set up as circles of the same size, and the coordinates of the next position of the robots are optimized by an improved particle swarm optimization, and multiple robots will move to these planned position points until they reach the end

point. The next positions of the robots, as well as the dynamic obstacles, are updated according to Equations (1) and (2):

$$x_i^{n+1} = x_i^n + v_i cos(\mu_i) \tag{1}$$

$$y_i^{n+1} = y_i^n + v_i sin(\mu_i) \tag{2}$$

where x_i^{n+1}, y_i^{n+1} are the x-coordinate and y-coordinate of the next position of mobile robot i v_i is the velocity of mobile robot i, which can be expressed in terms of the distance traveled at each step, and μ_i is the radial position of the mobile robot. As multiple robots are collaborating in path planning, the most important thing to consider is the length of the planned path and whether threats, such as static obstacles, dynamic obstacles, and other robots, are avoided, where the cost of moving path length cost is shown in Equations (3)–(5):

$$Cost_1 = \sum_{i=1}^{N} (f_i + g_i) \tag{3}$$

$$f_i = \sqrt{\left(x_i^{n+1} - x_i^n\right)^2 + \left(y_i^{n+1} - y_i^n\right)^2} \tag{4}$$

$$g_i = \sqrt{\left(x_i^{n+1} - x_i^{goal}\right)^2 + \left(y_i^{n+1} - y_i^{goal}\right)^2} \tag{5}$$

where (x_i^{goal}, y_i^{goal}) are the endpoint coordinates of robot i and N is the number of robots. The cost of avoiding static obstacles is shown in Equations (6) and (7):

$$Cost_2 = \begin{cases} \omega;\ if\ dis_i^s \le d_s \\ 0;\ if\ dis_i^s > d_s \end{cases} \tag{6}$$

$$dis_i^s = \sum_{i=1}^{N} \sum_{j=1}^{NS} \sqrt{\left(x_i^{n+1} - x_j^s\right)^2 + \left(y_i^{n+1} - y_j^s\right)^2} \tag{7}$$

where ω is the penalty term; dis_i^s is the sum of the distances between robot i and all static obstacles; NS is the number of static obstacles; (x_j^s, y_j^s) are the coordinates of static obstacle j, and d_s is the safety distance set by the model, expressed in terms of the robot's radius. The cost of avoiding dynamic obstacles is shown in Equations (8) and (9):

$$Cost_3 = \begin{cases} \omega;\ if\ dis_i^d \le d_s \\ 0;\ if\ dis_i^d > d_s \end{cases} \tag{8}$$

$$dis_i^d = \sum_{i=1}^{N} \sum_{j=1}^{ND} \sqrt{\left(x_i^{n+1} - x_j^d\right)^2 + \left(y_i^{n+1} - y_j^d\right)^2} \tag{9}$$

where dis_i^d is the sum of distances between robot i and all dynamic obstacles; ND is the number of dynamic obstacles, and (x_j^d, y_j^d) is the coordinates of dynamic obstacle j. The cost of avoiding other robots is shown in Equations (10) and (11):

$$Cost_4 = \begin{cases} \omega;\ if\ dis_i^r \le d_s \\ 0;\ if dis_i^r > d_s \end{cases} \tag{10}$$

$$dis_i^r = \sum_{i=1}^{N} \sum_{j=1}^{N-1} \sqrt{\left(x_i^{n+1} - x_j^r\right)^2 + \left(y_i^{n+1} - y_j^r\right)^2} \tag{11}$$

where (x_j^r, y_j^r) are the coordinates of the other robot. The final fitness function is expressed as the sum of the four costs and is calculated according to Equation (12):

$$Cost = Cost_1 + Cost_2 + Cost_3 + Cost_4 \tag{12}$$

The algorithm performance evaluation system proposed in this paper covers a number of key metrics, including the number of steps executed by the robot, the distance traveled, the deviation error of the path from the desired trajectory, the target distance yet to be reached, the total fitness, and the optimal fitness value. Among them, the average path deviation (APDE) is used to measure the degree of difference between the planned path and the ideal or optimal path. The ideal path planning route is a straight line from the start point to the end point of the robot, in which case, the length of the planned path is the shortest, which we call the ideal path, and the average path deviation represents the average value of the difference in distance between the planned path and the ideal path at each corresponding point. The size of the average path deviation reflects the accuracy and efficiency of the planning algorithm in finding the optimal path. If the average path deviation is small, it means that the difference between the path generated by the algorithm and the optimal path is not large, and the algorithm performs well. Conversely, if the average path deviation is large, it indicates that the algorithm has a large deviation in finding the optimal path, and further optimization of the algorithm or adjustment of the parameters may be required.

The average untraveled goal distance (AUGD) measures the average distance between the new position and the goal position produced by the algorithm at each step in the planning process. Specifically, when we use some kind of path planning algorithm to find the optimal path from the start point to the end point, the algorithm gradually generates a series of points or nodes as candidate points for the path. These points may not point directly to the goal point, but are generated in the process of gradually approaching the goal point. The significance of this metric is that it helps us understand the efficiency of the algorithm in planning the path. If the average untraveled goal distance is small, it means that the algorithm is able to get closer to the goal point at each step, so that the algorithm can find the optimal path faster. Conversely, if the average untraveled target distance is large, it means that the algorithm has a large deviation in the planning process, and more steps may be needed to find the optimal path.

3. Proposed Algorithm

In this section, the traditional particle swarm optimization, the motivation of the proposed algorithm, and the related strategies will be presented.

3.1. Particle Swarm Optimization

Particle swarm optimization is a kind of heuristic intelligent algorithm jointly proposed by American scholars Kennedy and Eberhart in 1995 [22], and its basic idea originates from the inspiration of the research results of modeling and simulation on the behavior of bird groups. Its core idea is to use the sharing of information by individuals in the group to make the movement of the whole group produce an evolution process from disorder to order in the problem-solution space, so as to obtain a feasible solution to the problem.

The particle swarm optimization is initialized as a group of random particles and then finds the optimal solution based on iterations. In each iteration, the particles update themselves by keeping track of two values: the first is the optimal solution found by

the particles themselves; the second is the optimal solution currently found by the entire population. The velocity of the particle is updated with Equation (13):

$$v_i = w \times v_{i-1} + c_1 r_1 (\ pbest_i - x_i) + c_2 r_2 (\ gbest_i - x_i) \tag{13}$$

where $pbest_i$ is the individual known optimal solution; $gbest_i$ is the population known optimal solution; w is the inertia weight; c_1 and c_2 are the learning factors that take the value of 2, and r_1 and r_2 are random numbers in the range of [0, 1]. The positions of the particles are updated according to the following Equation (14):

$$x_{i+1} = x_i + v_i \tag{14}$$

3.2. Motivation

The algorithm proposed in this paper is improved on the basis of particle swarm optimization, which as a classical swarm intelligence algorithm has the advantages of few parameters and is easy to implement, but at the same time, there are also problems, such as a tendency to fall into a local optimum, sensitive parameter selection, etc. In response to these problems, many scholars have improved the algorithm from different aspects.

For example, Zhou et al. [45] and others designed adaptive inertia weights to balance the exploration and development ability of particle swarm optimization. Chiheb et al. [46] and others proposed the MSPSO algorithm with adaptive factor selection, which uses adaptive strategies to dynamically select factors and enhance optimization performance; the proposed algorithm has a better convergence speed, and the performance of the algorithm is more stable. Liu et al. [47] and others proposed a multi-strategy MPSO algorithm with nonlinear inertia weights based on chaos, a stochastic learning strategy, a position update strategy, position update strategy, and other multi-strategy MPSOs, which enhance the ability of the particle swarm optimization to solve complex problems. Xing et al. [48] designed a new nonlinear inertia weight and a new fitness function to better deal with the problem of image segmentation. Liu et al. [49] proposed a particle swarm optimization algorithm, SDPSO, based on strategy dynamics, which introduces evolutionary game theory to control the population's performance through the selection mechanism and the mutation mechanism. Shami et al. [50] proposed the VPPSO algorithm, which puts forward the idea of velocity pause, allowing the particles to move at the same speed as the previous iteration, which better balances the ability of exploration and exploitation. Hu et al. [51] introduced the population center of gravity and the boundary rebound strategy in particle swarm optimization, which overcomes the traditional particle swarm optimization, which prematurely converges into the local optimum, and balances the algorithm's ability of global exploration and local exploitation by introducing a probabilistic method to update the inertia weights and other strategies. Lin et al. [52] proposed a hybrid algorithm culture–particle swarm optimization, which enhances optimization by integrating cultural evolution with swarm intelligence. Hsu et al. [53] combined the particle swarm optimization and whale optimization algorithm, which has a good performance in the field of material handling in container terminals. Gong et al. [54] proposed a quantum particle swarm optimization method based on diversity migration, which can indicate the direction of iterative population optimization and has a more stable convergence ability.

From the above review, it can be seen that many experts and scholars have improved the inertia weights of particle swarm optimization, and seldom made changes in the learning factor, but adjusting the learning factor can balance the algorithm's global exploration ability and local development ability to play a big role, so this paper introduces a fuzzy controller to dynamically adjust the learning factor, and at the same time, in order to reduce the impact of random initialization on the performance of the algorithm in the traditional

particle swarm optimization, this paper also introduces a fuzzy controller to dynamically adjust the learning factor. This paper also introduces exponential noise, and in order to better utilize the solution space, a dynamic balance mutation factor is introduced. These three strategies are described in detail in subsections.

3.3. Improved PSO Based on Multiple Strategies (FMEPSO)

The algorithm proposed in this paper is improved on the basis of the PSO algorithm, which has high flexibility and plasticity compared with other algorithms, so we choose the canonical PSO as the base algorithm by improving it in three aspects. In this study, we propose an improved particle swarm optimization, which further balances the algorithm's global exploration capability and local exploitation capability, while accelerating the convergence of the algorithm by introducing mechanisms such as fuzzy controllers, mutation operators, and exponential noise. The aim of this paper is to study the problem of multiple robots planning paths collaboratively, using the improved particle swarm optimization for path planning and considering the obstacle avoidance needs of static and dynamic obstacles.

3.4. Fuzzy Controller

In particle swarm optimization, the selection of individual cognitive and social learning factors plays a crucial role in the performance of the algorithm when the particles make positional decisions. These two factors often receive multiple influences, such as the change in the number of iterations of the algorithm, the speed of the particles, and so on.

In the initial stage of the algorithm iteration, setting a larger individual cognitive factor and a smaller social learning factor can help particles to rely more on their own experience and search, thus exploring the local space faster and searching for possible local optimal solutions. This setting helps to quickly locate potential solutions in the search space. Gradually decreasing the individual cognitive factor and increasing the social learning factor as the iteration proceeds helps the particle to rely more on the experience and information of other particles in the group during the search. This can motivate the particles to explore the whole search space more extensively, jumping out of the local optimal solution and approaching the global optimal solution. Increasing the social learning factor can enhance information sharing and communication among particles, which can help the whole particle group search for the optimal solution more effectively and collaboratively. By dynamically adjusting the individual cognitive factor and the social learning factor, the effects of global and local search can be balanced at different stages of the algorithm, thus improving the performance of the algorithm and making it more effective in finding the optimal solution in the complex search space.

The influence of particle speed on the learning factor is also the same; in the particle swarm optimization algorithm, when the particle speed is small, it is suitable to set a larger individual cognitive factor and a smaller social learning factor, which helps the particles to rely more on their own experience and information; when the particle speed increases, it is suitable to reduce the individual cognitive factor and increase the social learning factor, which can strengthen the information sharing and communication between particles and help the particles make better use of group intelligence, accelerate the global search process, and help jump out of the local optimal solution and approach the global optimal solution. Such a setting helps to balance the global search and local search to improve the performance of the algorithm.

Inspired by this, this study introduces a fuzzy control strategy to adjust the individually learnt cognitive factor and the socially learnt factor in the particle swarm optimization. At different periods of the algorithm, the fuzzy control will change individual cognitive

and social learning factors with the number of iterations of the algorithm and the speed of the particles.

The fuzzy controller proposed in this paper includes two inputs and two outputs, in which the current iteration number of the population is taken as input 1, and the fuzzy sets include 'Low', 'Medium', and 'High'. The speed of the population is taken as input 2, and the fuzzy set includes 'Fast', 'Medium', and 'Slow'. Individual awareness factor and social learning factor of the particles are taken as two outputs, and the fuzzy sets are represented by 'Low', 'MediumLow', 'MediumHigh', and 'High'. The number of iterations of the algorithm and the particle velocity have the same affiliation function, as shown in Figure 1, and the individual cognitive factor and the social learning factor have the same affiliation function, as shown in Figure 2.

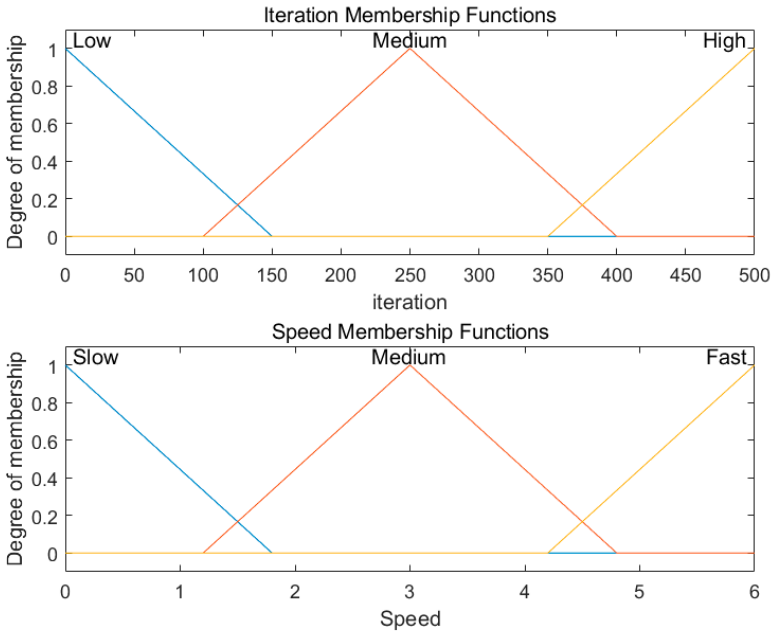


Figure 1. Fuzzy controller inputs—number of iterations and speed of affiliation function.

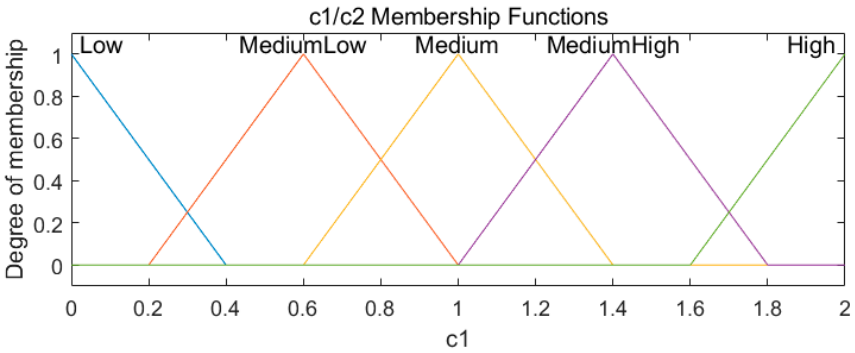


Figure 2. Fuzzy controller output—c_1/c_2 affiliation function.

Based on this fuzzy controller, the following IF-THEN fuzzy rules and rules tables: Tables 1 and 2 are formulated in this paper:

- Rule 1: IF number of iterations = 'Low' AND speed = 'Slow' THEN c_1 = 'High' AND c_2 = 'Low';
- Rule 2: IF number of iterations = 'Medium' AND speed = 'Slow' THEN c_1 = 'MediumHigh' AND c_2 = 'MediumLow';
- Rule 3: IF number of iterations = 'High' AND speed = 'Slow' THEN c_1 = 'Medium' AND c_2 = 'Medium';

- Rule 4: IF number of iterations = 'Low' AND speed = 'Medium' THEN c_1 = 'MediumHigh' AND c_2 = 'High';
- Rule 5: IF number of iterations = 'Medium' AND speed = 'Medium' THEN c_1 = 'Medium' AND c_2 = 'Medium';
- Rule 6: IF number of iterations = 'High' AND speed = 'Medium' THEN c_1 = 'MediumLow' AND c_2 = 'MediumHigh';
- Rule 7: IF number of iterations = 'Low' AND speed = 'Fast' THEN c_1 = 'Medium' AND c_2 = 'Medium';
- Rule 8: IF number of iterations = 'Medium' AND speed = 'fast' THEN c_1 = 'MediumLow' AND c_2 = 'MediumHigh';
- Rule 9: IF number of iterations = 'High' AND speed = 'Fast' THEN c_1 = 'Low' AND c_2 = 'High'.

Table 1. Fuzzy rules: impact of number of iterations and speed on c_1.

Number of Iterations \ Speed	Slow	Medium	Fast
Low	High	MediumHigh	Medium
Medium	MediumHigh	Medium	MediumLow
High	Medium	MediumLow	Low

Table 2. Fuzzy rules: impact of number of iterations and speed on c_2.

Number of Iterations \ Speed	Slow	Medium	Fast
Low	Low	High	Medium
Medium	MediumLow	Medium	MediumHigh
High	Medium	MediumHigh	High

3.5. Dynamic Balance of Mutation Factor

Since the PSO has the disadvantage of easily falling into local optimal solutions and not being able to fully search the solution space, this paper introduces a new mutation factor into the PSO, and there have been a number of researchers previously generating the mutation factor through the use of a random factor; this paper, based on this, also takes the particles with the top three fitness values as the leader particles during each iteration, and the reason for retaining the better-fitting individuals is to accelerate the convergence speed of the algorithm so that the population evolves faster toward a better solution, and to avoid over-exploring the poorly adapted regions in the search space, so as to focus more on exploring in depth the regions that may contain a better solution.

By randomly using one of the particles in the leader factor and two random particles together to generate a mutation factor, the introduction of this mutation factor adds a certain degree of randomness and diversity to the PSO, which helps to explore the space of possible solutions more extensively in the search space, thus improving the global search capability of the algorithm.

The calculation of the mutation factor is shown in Equation (15):

$$x_{new} = r * x_{leader} + (1 - r)(x_{rand1} - x_{rand2}) \quad (15)$$

where r is used as a scaling factor and is updated using Equation (16):

$$r = \begin{cases} max(r * (1 - 0.1 * \frac{fit_{current}}{fit_{prev}}), r_{min} & if \ \ no\_improvecount \geq 3 \\ min(r * (1 + 0.1 * \frac{fit_{prev}}{fit_{current}}), r_{max} & if \ \ improvecount \geq 3 \end{cases} \quad (16)$$

where $fit_{current}$ denotes the best fitness value of the current run; fit_{prev} prev denotes the best fitness value of the run before this; no_improvecount denotes the counter that records the continuous failure of the particle's fitness value to improve; if the fitness value does not improve for three consecutive times, at this time, the value of r will decrease, reducing the influence of the elite individuals, increasing the exploratory nature, and hopefully, jumping out of the local optimum. If the fitness value is updated three consecutive times in a row, the value of r will rise, accelerating the search process by increasing the influence of the elite individuals on the mutation factor and increasing the exploitation capability of the algorithm. These two formulas respond to the changing needs of the algorithm for exploration and exploitation at different stages of the process by dynamically adjusting the value of r. When the algorithm performs poorly, exploration is increased to find new possible solutions; when the algorithm performs well, exploration is reduced to make more use of the current good solutions. Such a design helps to balance exploration and exploitation, accelerate the convergence of the algorithm, and improve the overall performance of the algorithm.

3.6. Exponential Noise

Exponential noise is a specific type of random noise, which is characterized by the exponential distribution of noise values and is widely used in random signal processing and optimization algorithms. In this paper, exponential noise is introduced into the PSO, used to update the matrix of particles during the iteration process, and after every 50 iterations, the exponential noise is used to randomly update the sequence of the population, so as to make the distribution of the population more uniform, and at the same time, reduce the impact of random initialization on the particle swarm optimization, using exponential noise for particle position update, as shown in Equations (17) and (18):

$$Q = \alpha * log(1 - rand(P * K)) \quad (17)$$

$$x_i = 0.5 * (ub - lb) * Q_i + x_{sp} * H_i \quad (18)$$

where Q is the generated exponential noise sequence, taking the value of; H is a matrix, which takes the value range of $[-1, 1]$; P is the number of particles; K is the problem dimension, and α is a scaling factor. The particle swarm optimization with the introduction of exponential noise enhances the randomness, helps to avoid the algorithm from falling into local optimal solutions, makes better use of the solution space, and improves the global search capability of the algorithm.

3.7. Pseudocode and Flowchart of the Algorithm

The algorithm inputs parameters such as population size, number of iterations, dimensionality, maximum velocity, and maximum and minimum inertia weights and outputs the deviation error of the expected trajectory (APDE), the unattained goal distance (AUGD), the fitness value and its sum, the number of robot steps moved and the length of the planned path. Firstly, the algorithm initializes each parameter and population, then iterates the number of iterations and each particle. Firstly, the values of c_1 and c_2 are calculated by the fuzzy controller, and then, the fitness values of the particles and the next position of the particles are calculated, and if the number of iterations can be divisible by 50, exponential

noise is introduced to perform the updating of the particle's position again, and if it cannot be divisible by 50, then mutant particles are introduced. If the fitness value of the mutant particle is better than the best fitness value, then the particle replacement is performed. The pseudocode of Algorithm 1 are shown below, and the flowchart of the algorithm in the below shows the logic of the whole algorithm more intuitively.

Algorithm 1: FMEPSO

Input: *N (Population size), T (The number of iterations), d (Dimension), Vmax (Velocity),*w_{max}, w_{min}
Output: ***APDE, AUGD, total_fitness, fitness, step, distance, bestcost***
1: Initialize the parameter and population
2: ***For*** *t* = 1*: T*
3: ***For*** *i* = 1*: N*
4: Calculate the values of c_1 and c_2 using a fuzzy controller
5: Update the position of the individual according to the Equations (1) and (2)
6: **If** mod(t, 50) = 0
8: Update the position according to the Equations (17) and (18)
9: **Else**
10: Add mutation factors according to Equations (15) and (16)
11: **End if**
12: **If** the fitness value is improved
13: Update the no_improve count
14: **Else**
15: Update the improve count
16: **End if**
16: ***End for***
17: ***End for***

The flowchart of the algorithm is shown in Figure 3:

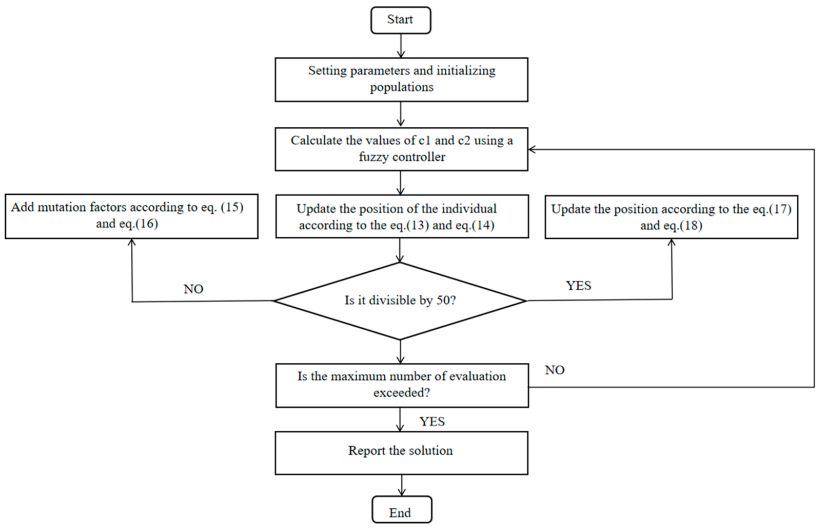


Figure 3. The flowchart of the algorithm.

4. Experimental Analysis

In this subsection, in order to verify the reasonableness and practicality of the algorithm, the algorithm is simulated and experimented on three kinds of scenarios, namely, simple scenario, complex scenario, and random scenario, where the dynamic obstacles

have circles with the same radius and move between the lines, connecting the specified start and end points, and the static obstacles are represented by the circles with different radius in the simple scenario, the complex scenario, and the random scenario, in which the dynamic obstacles and the number of static obstacles and robots are shown in Table 3, all of which are of size 100 × 100:

Table 3. Number of obstacles and robots for the three maps.

	Simple Scenario	Complex Scenario	Random Scenario
Dynamic obstacles	3	4	3
Static obstacles	7	10	10
Number of Robots	6	7	7

In order to verify the superiority of the algorithm proposed in this paper, this paper compares this algorithm with eight algorithms, such as SDSCA, EAPSO, VPPSO, MPSO, PSO, WOA, HHO, and SCA. Each algorithm is run independently for 20 times in each kind of scenarios, in which the maximum number of iterations for the simple scenarios is 150; the number of populations is 30; the maximum number of iterations for the complex scenarios and random scenarios is 500; the number of populations is 150, and the best value, the worst value, and the average value of the best fitness value in 20 times of path planning for each algorithm are calculated. The internal parameters set by each algorithm are shown in Table 4. The fitness values of each algorithm in each scenario are shown in Table 5.

Table 4. Parameter settings for each algorithm.

Algorithm	Parameter
FMEPSO	Vmax = 6, wmax = 0.6, wmin = 0.2
SDSCA	a = 2, F = 0.8, CR = 0.95
VPPSO	c_1 = 2, c_2 = 2;
PSO	Vmax = 6, wMax = 0.6, wMin = 0.2, c_1 = 2, c_2 = 2;
WOA	b = 1
HHO	beta = 1.5
SCA	a = 2

As can be seen from Table 5, in the simple scenario, the best, worst, and average of the best fitness values of FMEPSO are the best among the nine algorithms, but in terms of variance, FMEPSO is slightly worse than EAPSO and SCA, indicating that there is still room for improvement in the algorithm proposed in this paper in terms of its stability. In a complex scenario, the best cost of FMEPSO is also better than the other eight algorithms, which is slightly better than the best fitness value of VPPSO, and the variance of FMEPSO is also the smallest on complex maps, which shows that FMEPSO has a better performance in complex environments, and the algorithm is also more stable. In a random scenario, FMEPSO also performs very well; among the nine algorithms, the best value, the worst value, and the average fitness value are the best, and the variance is also the smallest, which shows that FMEPSO algorithms have the ability to cope with different maps randomly and have better stability. In order to exclude the effect of special extremes, we plotted box plots of the 20 final fitness values for each algorithm, as shown in Figure 4, and with the help of variance, we can find that the FMEPSO algorithm proposed in this paper has better stability.

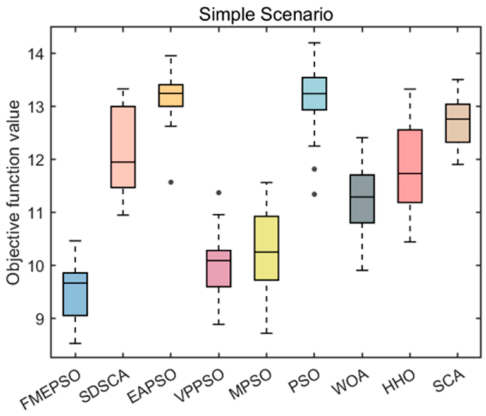
Table 5. Comparison of best cost values and Wilcoxon rank sum test for each algorithm.

		FMEPSO	SDSCA	EAPSO	VPPSO	MPSO	PSO	WOA	HHO	SCA
simple scenario	worst	**10.4641**	13.3307	13.9564	11.3726	11.5637	14.2005	12.4109	13.3275	13.5077
	best	**8.5281**	10.9508	11.5701	8.8868	8.7197	11.3428	9.9067	10.4439	11.9045
	mean	**9.5128**	12.1293	13.1727	10.0368	10.2841	13.1254	11.2782	11.8542	12.7094
	Var	**0.3149**	0.6125	0.2486	0.4213	0.6875	0.5426	0.4770	0.7467	0.2458
	p		6.80×10^{-8} (+)	6.80×10^{-8} (+)	1.14×10^{-2} (+)	4.32×10^{-3} (+)	6.80×10^{-8} (+)	1.66×10^{-7} (+)	7.89×10^{-8} (+)	6.80×10^{-8} (+)
complex scenario	worst	**11.5534**	15.04	15.9519	12.7046	13.654	15.7279	12.8756	15.5984	15.8003
	best	**9.3624**	12.2884	14.1494	9.9195	10.2686	14.2312	9.8156	11.2598	13.3518
	mean	**10.5642**	13.8259	15.1023	11.4575	11.7055	14.9368	11.8420	12.7894	14.8008
	Var	**0.2486**	0.5205	0.2887	0.6451	1.0870	0.1743	0.6220	1.2486	0.5374
	p		6.79×10^{-8} (+)	6.79×10^{-8} (+)	2.80×10^{-3} (+)	3.70×10^{-5} (+)	6.79×10^{-8} (+)	1.25×10^{-5} (+)	9.16×10^{-8} (+)	6.79×10^{-8} (+)
random scenario	worst	**12.6813**	15.9934	15.9358	13.8330	17.8904	17.9774	14.9850	19.7673	16.8776
	best	**10.5654**	11.5226	12.5795	10.9539	11.0563	14.2204	11.6118	12.9162	13.7253
	mean	**11.6657**	14.4022	14.7626	12.4355	13.6077	15.8202	13.4639	15.8884	15.7820
	Var	**0.3182**	1.8380	1.0531	1.2350	3.7087	0.9069	0.8128	3.4326	0.5877
	p		7.95×10^{-7} (+)	9.16×10^{-8} (+)	5.31×10^{-2} (=)	4.60×10^{-4} (+)	6.80×10^{-8} (+)	9.13×10^{-7} (+)	6.80×10^{-8} (+)	6.80×10^{-8} (+)

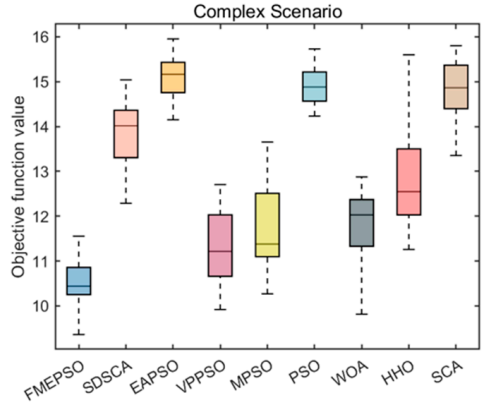
Simply looking at the best fitness value does not fully show the superiority of the FMEPSO algorithm; for this reason, this paper also counts the total_fitness, APDE, and AUGD of the individual algorithms in each map in 20 operations, and the values of the three indices are shown in Table 6. From the table, it can be seen that in all three scenarios, FMEPSO's total fitness is the smallest, and the best cost central score is not as good as the individual algorithms, but total fitness reflects the overall cost of the robot in the process of moving, which is the best of all algorithms, indicating that the algorithm can better meet the predetermined goals, and the quality of the overall path planning is better. Also, considering APDE and AUGD of the three scenarios, FMEPSO is also the lowest among the nine algorithms, which indicates that the algorithm proposed in this paper has a small degree of deviation between the actual paths and the preset optimal paths when performing path planning, and generates paths with high efficiency.

The number of steps made by the robot and the total length of the actual generated paths are also important indicators for considering how good the generated paths are. In this paper, the number of steps made and the path length of each robot in the three scenarios are averaged by running 20 experiments, and the statistical results are shown in the following table.

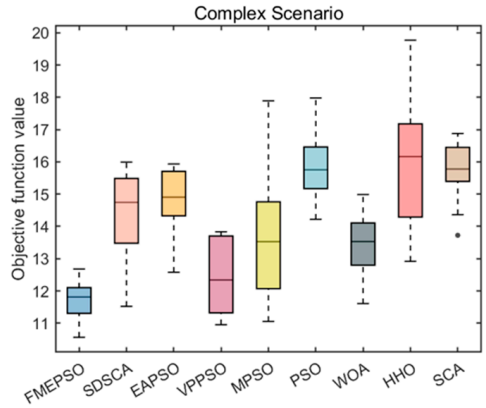
From the data in Tables 7 and 8, it can be seen that FMPSO performs much better than SDSCA, EAPSO, PSO, WOA, HHO, and SCA, and slightly better than VPPSO and MPSO, in terms of the total number of steps made by the robots, the length of the planned paths, and the performance of the movement of a single robot. FMEPSO performs slightly worse than VPPSO and MPSO in terms of the performance of the individual robots. The bolded values in the table are the best performance values for each robot among the nine algorithms. It can be seen that the majority of the best values are concentrated in FMEPSO, and the total values for both the number of steps and the path length are the smallest in the three scenarios.



(**a**) simple scenario



(**b**) complex scenario



(**c**) Random scenario

Figure 4. Box plots of the objective function of each algorithm under the three scenarios.

In order to visualize the smoothness of the paths planned by each algorithm, Figures 5–7 show the sample paths planned by different algorithms in the three scenarios. The grey circles are static obstacles; the dark blue circles are dynamic obstacles; the light blue denotes the mobile robot, and the 'x' denotes the end point of the robot and the dynamic obstacles. In the simple scenario, we can see that the path planned by FMEPSO is the smoothest, especially compared to VPPSO and MPSO, where multiple robots tend to meet. In the path optimized by the FMEPSO algorithm, the robots are able to bypass the obstacles better, and in the complex environments, in terms of path smoothing, the path planning of FMEPSO is better than that of EAPSO, PSO, WOA, HHO, SDSCA, and SCA algorithms, slightly better than VPPSO when obstacles are bypassed, and the path smoothness of MPSO is basically the same in a random environment. We can see that in the line without obstacles blocking the line, the line planned by FMEPSO runs straight,

which is basically the same as the ideal line, with a small deviation of the path, and the lines planned by VPPSO and MPSO algorithms plan routes that are still somewhat curved when there are no obstacles blocking the route, and the smoothness of the other algorithms is even worse.

Table 6. Comparison of Total_fitness, APDE, and AUGD for 9 algorithms in different scenarios.

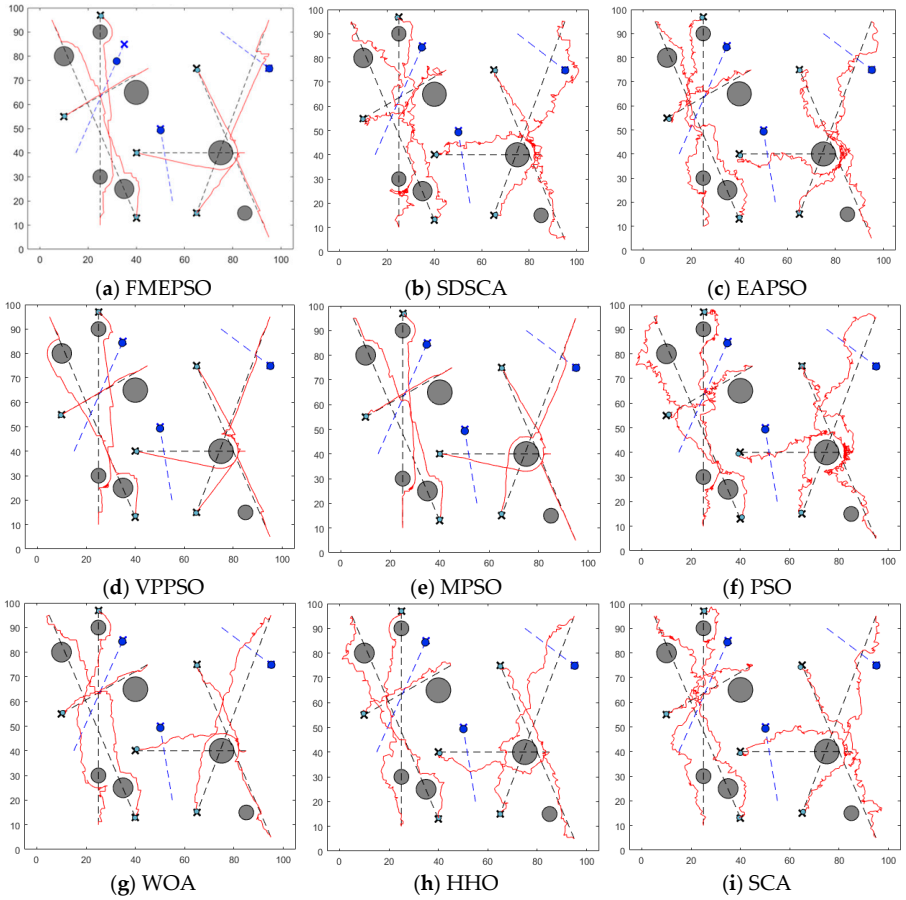
Algorithm		FMEPSO	SDSCA	EAPSO	VPPSO	MPSO	PSO	WOA	HHO	SCA
simple scenario	total_fitness	1.64×10^4	2.46×10^4	3.87×10^4	1.64×10^4	1.67×10^4	3.38×10^4	1.93×10^4	2.29×10^4	3.16×10^4
	APDE	46.1918	296.3131	485.4991	63.9312	63.4397	440.9328	137.7289	237.5415	461.4975
	AUGD	1.52×10^4	2.35×10^4	2.97×10^4	1.56×10^4	1.59×10^4	2.80×10^4	1.84×10^4	2.18×10^4	2.96×10^4
complex scenario	total_fitness	9.44×10^3	1.53×10^4	3.08×10^4	1.05×10^4	9.83×10^3	2.22×10^4	1.20×10^4	1.47×10^4	1.89×10^4
	APDE	27.0080	213.6326	364.6021	54.4634	42.1172	282.9601	75.6612	175.8038	307.5153
	AUGD	8.75×10^3	1.42×10^4	2.27×10^4	9.67×10^3	9.11×10^3	1.81×10^4	1.06×10^4	1.29×10^4	1.71×10^4
random scenario	total_fitness	9.95×10^3	1.96×10^4	2.57×10^4	1.06×10^4	1.29×10^4	2.08×10^4	1.17×10^4	1.64×10^4	1.95×10^4
	APDE	22.4036	297.6657	374.8363	69.4552	45.9525	379.8032	104.0334	258.5672	361.1101
	AUGD	8.71×10^3	1.51×10^4	1.76×10^4	9.50×10^3	9.60×10^3	1.78×10^4	1.02×10^4	1.46×10^4	1.73×10^4

Table 7. Comparison of different robot steps and sums for 9 algorithms in different scenarios.

		FMEPSO	SDSCA	EAPSO	VPPSO	MPSO	PSO	WOA	HHO	SCA
simple scenario	Robot_1	**86.40**	125.30	163.20	88.00	88.95	146.75	110.40	129.00	146.10
	Robot_2	**81.95**	121.90	150.95	87.80	89.75	145.45	91.70	110.75	153.25
	Robot_3	32.95	54.45	70.65	**32.45**	32.65	70.80	40.00	45.05	66.45
	Robot_4	82.20	113.65	140.70	75.25	**81.00**	132.50	95.75	111.45	139.85
	Robot_5	**42.90**	68.40	88.80	46.45	48.00	84.65	51.95	60.30	89.00
	Robot_6	64.45	101.60	121.60	**62.90**	65.00	118.40	75.55	90.00	125.45
	Total	**390.85**	585.30	735.90	392.85	405.35	698.55	465.35	546.55	720.10
complex scenario	Robot_1	**47.40**	68.50	83.25	56.85	55.70	73.10	60.45	77.50	83.60
	Robot_2	**72.60**	99.90	161.25	77.05	75.95	133.40	83.25	94.60	113.60
	Robot_3	**26.25**	35.10	46.70	26.30	29.35	41.45	27.65	35.55	42.50
	Robot_4	**23.25**	31.50	46.10	24.60	28.50	43.60	27.95	38.00	39.55
	Robot_5	59.00	87.80	91.90	59.20	**56.65**	88.70	64.50	80.10	103.35
	Robot_6	55.00	79.05	90.65	**54.60**	57.90	81.45	61.05	71.25	91.90
	Robot_7	**17.15**	48.90	45.45	19.10	19.40	37.85	24.20	31.90	48.85
	Total	**300.65**	450.75	565.30	317.70	323.45	499.55	349.05	428.90	523.35
random scenario	Robot_1	50.10	91.05	90.05	53.00	**49.05**	90.10	66.90	94.65	84.50
	Robot_2	**75.85**	117.90	129.25	96.55	77.45	129.75	84.10	106.05	135.50
	Robot_3	**25.85**	44.40	54.10	26.30	27.15	56.60	33.35	42.90	54.25
	Robot_4	26.65	34.80	57.50	26.05	**25.95**	53.40	32.95	42.90	41.55
	Robot_5	**58.70**	107.45	112.50	60.15	70.60	116.25	72.20	96.40	116.10
	Robot_6	**55.65**	88.80	106.80	57.15	57.80	109.30	67.70	95.85	108.30
	Robot_7	11.65	31.65	24.80	**11.40**	11.60	23.65	16.25	20.85	26.90
	Total	**304.45**	516.05	575.00	330.60	319.60	579.05	373.45	499.60	567.10

Table 8. Comparison of distance traveled and sum of different robots for 9 algorithms in different scenarios.

		FMEPSO	SDSCA	EAPSO	VPPSO	MPSO	PSO	WOA	HHO	SCA
simple scenario	Robot_1	**102.57**	153.91	202.10	108.59	105.67	181.41	131.91	154.38	179.80
	Robot_2	**97.99**	150.01	186.41	107.89	105.67	179.36	110.49	133.97	188.00
	Robot_3	**40.35**	67.28	86.81	40.70	40.29	87.39	48.98	55.15	81.84
	Robot_4	98.13	139.77	173.87	**94.04**	98.11	163.83	115.10	134.39	171.77
	Robot_5	**51.89**	83.65	109.57	57.26	56.38	105.01	63.07	73.38	109.52
	Robot_6	**78.33**	124.77	149.80	78.52	79.35	147.00	91.25	109.34	153.63
	Total	**469.26**	719.38	908.57	487.00	485.48	864.00	560.80	660.61	884.56
complex scenario	Robot_1	**57.24**	83.65	101.85	67.61	63.55	90.07	70.68	90.46	102.49
	Robot_2	**86.11**	121.97	197.66	94.82	89.28	163.32	96.32	110.80	139.98
	Robot_3	**31.55**	42.95	57.48	33.12	34.74	51.50	33.48	42.87	52.36
	Robot_4	**30.66**	38.74	56.67	30.90	31.11	53.75	49.36	45.67	48.67
	Robot_5	70.43	107.15	113.62	72.67	**68.40**	111.17	75.84	93.34	126.49
	Robot_6	**66.92**	96.67	111.75	67.55	69.31	100.62	72.30	85.22	113.68
	Robot_7	**19.65**	70.20	55.58	22.67	21.10	46.50	28.76	37.64	58.88
	Total	**362.58**	561.34	694.61	389.34	377.49	616.92	426.74	505.99	821.88
random scenario	Robot_1	58.69	111.35	111.76	64.65	**58.61**	111.43	78.53	112.13	104.21
	Robot_2	**87.68**	144.69	159.52	116.85	91.85	159.98	99.49	125.65	166.17
	Robot_3	**31.03**	54.60	66.63	32.43	32.87	70.25	38.75	51.74	66.53
	Robot_4	31.93	43.15	71.05	32.50	**31.23**	66.27	39.28	51.36	51.32
	Robot_5	**68.93**	131.19	138.54	74.16	82.97	143.54	84.47	114.11	142.67
	Robot_6	**65.74**	109.48	131.94	70.30	69.81	134.96	78.88	113.96	132.98
	Robot_7	**13.98**	38.78	30.96	14.14	14.18	28.95	20.20	25.20	32.80
	Total	**357.98**	633.24	710.41	405.03	81.52	715.38	439.61	594.14	696.68



(**a**) FMEPSO (**b**) SDSCA (**c**) EAPSO (**d**) VPPSO (**e**) MPSO (**f**) PSO (**g**) WOA (**h**) HHO (**i**) SCA

Figure 5. Simulation of online paths for each algorithm in simple scenario.

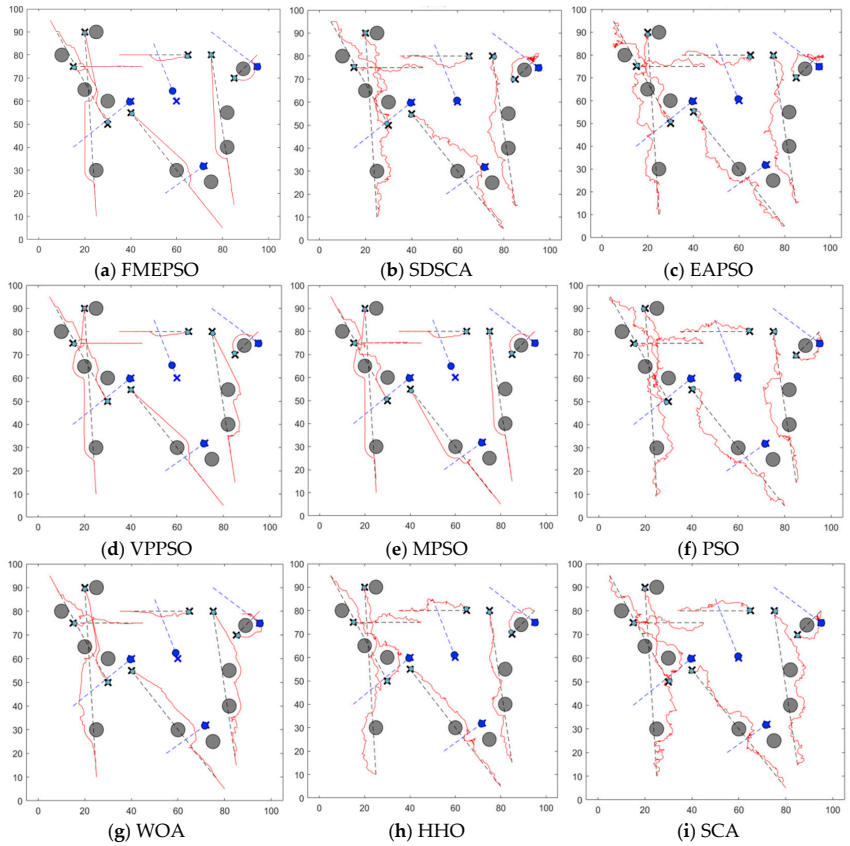


Figure 6. Simulation of online paths for each algorithm in complex scenario.

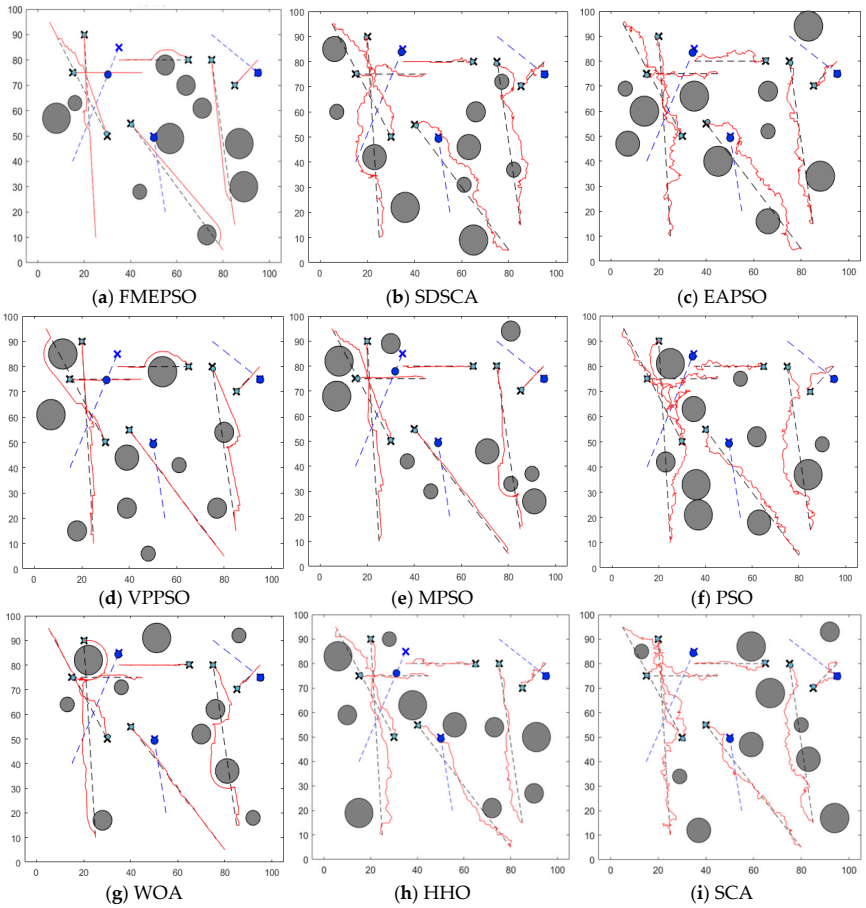
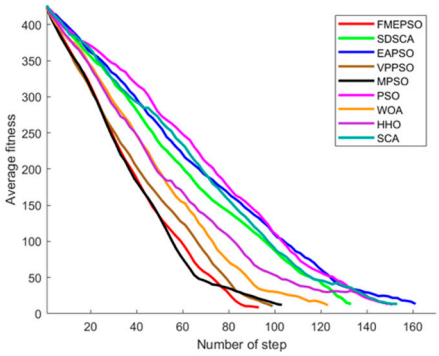
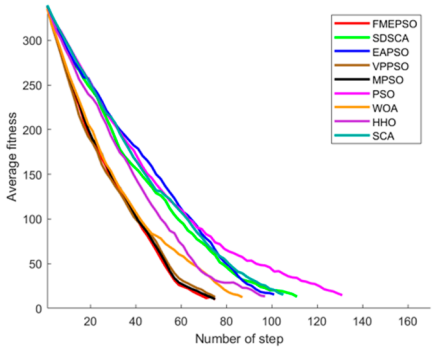


Figure 7. Simulation of online paths for each algorithm in random scenario.

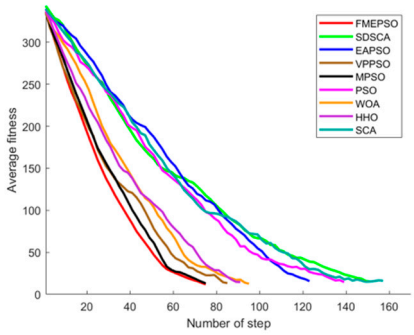
In order to allow the robot to plan the correct path in time when dealing with a situation such as a disaster, this paper verifies the convergence of nine algorithms, including FMEPSO. Figure 8 shows the convergence graph of each algorithm in three scenarios. From the convergence diagram, we can see that in simple scenarios, the convergence of FMEPSO is not as fast as that of the MPSO algorithm in the early iteration period, but the speed of the later iterations is better than that of MPSO. In complex scenarios, the iteration speeds of FMEPSO, MPSO, and VPPSO are basically the same, which is better than that of the other six algorithms, and in random scenarios, the convergence speed of FMEPSO is the best.



(**a**) Simple scenario



(**b**) Complex scenario



(**c**) Random scenario

Figure 8. Convergence effect of each algorithm under three scenarios.

In order to demonstrate the effectiveness of the overall strategy of the algorithm, this paper conducts ablation experiments on each strategy of the algorithm in both simple and complex environments, where the algorithm without fuzzy controller is called MEPSO; the algorithm without mutation factor is called FEPSO; the algorithm without exponential noise strategy is called FMPSO; the algorithm with only fuzzy controller is called FPSO; the algorithm with only mutation factor is called MPSO, and the algorithm with only exponential noise is called EPSO. The optimal fitness values of each algorithm, APDE,

AUGD, TOTAL FITNESS, as well as the number of steps and the length of the paths, are shown in Tables 9–12.

Table 9. Comparison of adaptation values and Wilcoxon rank sum test for ablation experiments.

		FMEPSO	MEPSO	FEPSO	FMPSO	FPSO	MPSO	EPSO
simple scenario	worst	10.4641	12.9586	12.3699	12.2946	11.9536	12.4596	12.3965
	best	8.5281	10.2368	10.2589	10.2566	9.1444	10.0152	10.1586
	mean	9.5128	11.0310	11.2521	11.0081	10.4826	11.1110	11.1727
	Var	0.3149	0.6739	0.5618	0.5331	0.6083	0.4425	0.6872
	p		2.96×10^{-7}	2.96×10^{-7}	5.21×10^{-7}	1.61×10^{-4}	1.92×10^{-7}	6.90×10^{-7}
complex scenario	worst	11.5534	12.7761	12.7837	13.4692	11.8312	13.5349	13.6869
	best	9.3624	10.2088	10.5363	11.0697	10.2993	11.2684	10.3422
	mean	10.5642	11.0869	11.5982	11.8679	11.1638	12.2818	11.9043
	Var	0.2486	0.4933	0.6869	0.6794	0.3700	0.8210	1.1632
	p		1.73×10^{-2}	7.65×10^{-4}	7.69×10^{-4}	5.80×10^{-3}	1.52×10^{-4}	1.45×10^{-3}

Table 10. Comparison of Total_fitness, APDE, and AUGD for ablation experiments.

		FMEPSO	MEPSO	FEPSO	FMPSO	FPSO	MPSO	EPSO
simple scenario	total_fitness	1.64×10^4	1.74×10^4	1.71×10^4	1.83×10^4	1.74×10^4	1.98×10^4	1.78×10^4
	APDE	46.1918	73.04	69.47	70.31	75.50	72.31	83.04
	AUGD	1.52×10^4	1.67×10^4	1.65×10^4	1.66×10^4	1.68×10^4	1.66×10^4	1.71×10^4
complex scenario	total_fitness	9.44×10^3	1.24×10^4	1.59×10^4	1.70×10^4	1.02×10^4	1.14×10^4	1.07×10^4
	APDE	27.0080	54.03	45.91	40.66	50.58	46.78	51.26
	AUGD	8.75×10^3	1.06×10^4	1.22×10^4	9.28×10^3	9.52×10^3	9.43×10^3	9.95×10^3

Table 11. Comparison of robot steps for ablation experiments.

		FMEPSO	MEPSO	FEPSO	FMPSO	FPSO	MPSO	EPSO
simple scenario	Robot_1	**86.40**	87.58	87.84	87.89	87.84	88.37	89.74
	Robot_2	**81.95**	91.11	90.32	89.00	91.16	90.26	93.42
	Robot_3	**32.95**	34.26	33.11	33.47	33.26	33.47	34.21
	Robot_4	**82.20**	89.26	87.11	86.37	86.47	86.47	89.26
	Robot_5	**42.90**	49.47	48.84	58.32	59.47	55.79	55.95
	Robot_6	**64.45**	66.95	66.68	66.32	67.63	68.11	67.79
	Total	**390.85**	418.63	413.89	421.37	425.84	422.47	430.37
complex scenario	Robot_1	**47.40**	59.80	52.60	54.20	55.50	56.56	60.89
	Robot_2	**72.60**	83.90	76.20	75.90	79.70	79.78	81.89
	Robot_3	**26.25**	26.30	27.80	27.60	28.10	28.22	27.67
	Robot_4	**23.25**	26.70	26.40	26.30	26.50	26.67	25.78
	Robot_5	**59.00**	60.40	61.20	61.70	60.70	61.33	63.67
	Robot_6	**55.00**	56.50	55.90	56.50	56.80	56.67	56.78
	Robot_7	**17.15**	20.50	25.50	18.70	24.60	21.33	20.44
	Total	**300.65**	334.10	325.60	320.90	331.90	330.56	337.11

Table 12. Comparison of distance traveled and sum for ablation experiments.

		FMEPSO	MEPSO	FEPSO	FMPSO	FPSO	MPSO	EPSO
simple scenario	Robot_1	**102.57**	105.35	104.39	103.00	151.75	103.01	104.96
	Robot_2	**97.99**	106.62	106.12	103.51	106.13	104.78	108.63
	Robot_3	**40.35**	41.04	40.96	40.52	40.31	40.14	41.38
	Robot_4	**98.13**	104.84	103.51	101.15	101.40	97.07	105.63
	Robot_5	**51.89**	58.39	57.51	65.80	66.24	63.38	64.95
	Robot_6	**78.33**	80.21	80.32	79.40	81.34	80.87	80.98
	Total	**469.26**	496.44	492.82	493.37	547.17	489.24	506.54
complex scenario	Robot_1	**57.24**	66.92	60.37	61.50	63.21	63.53	65.11
	Robot_2	**86.11**	97.45	88.69	89.52	93.61	92.24	94.22
	Robot_3	**31.55**	31.92	33.04	32.43	33.21	33.05	32.95
	Robot_4	**30.66**	30.98	31.25	31.24	30.93	30.92	31.02
	Robot_5	**70.43**	70.91	72.41	72.85	71.11	71.45	73.40
	Robot_6	**66.92**	67.99	67.43	68.02	67.76	67.29	67.54
	Robot_7	**19.65**	22.82	28.29	20.68	26.45	23.66	22.57
	Total	**362.58**	388.99	381.47	376.23	386.28	382.13	386.81

From the ablation experiments, it can be seen that FMEPSO has a better optimization than the other five algorithms; FMEPSO has the best optimization in all three environments, and also, this algorithm is more stable. From the point of view of the number of steps made by the robot, the number of steps made by the robot under the operation of the FMEPSO algorithm is also the smallest, and the planned path is also the shortest. Comprehensive analysis of the above shows that the advantages of FMEPSO in path planning are still more obvious; the combination of the three strategies also has a better effect and a higher value.

5. Conclusions

In order to plan the movement paths of multiple robots while avoiding all obstacles, this paper proposes an improved particle swarm optimization based on multiple strategies, such as a fuzzy controller. In this paper, a fuzzy controller is designed for dynamically adjusting the values of c_1 and c_2, while strategies, such as dynamically balancing the mutation factor and exponential noise, aim to make full use of the solution space, balancing global exploration and local exploitation while accelerating the convergence speed of the algorithm when the algorithm performs well. By conducting ablation experiments in both simple and complex scenarios, it is demonstrated that the strategies complement each other in the algorithm proposed in this paper, and the multi-strategy fusion shows the best algorithm performance. Simulation experiments are also conducted in three scenarios and compared with the other eight algorithms. It can be concluded that the algorithm proposed in this paper has a better performance in multi-robot path planning, as it is able to plan a path that completely avoids obstacles, has a short path length, and has fewer and smoother moving steps. Given the good performance of this algorithm in this paper, we can use it to solve the robot path planning problem in difficult problem and have superiority in solving multi-robot cooperative path planning and good practicality in many fields, such as logistics and distribution, industrial automation operation, and so on. However, the time complexity of the proposed algorithm still needs to be reduced. In terms of the path planning model, in order to better cope with the actual situation and make the algorithm more practical, it is also possible to set up irregular model obstacles and, at the same time, set up relevant constraints to make the path smoother. In the future, we will continue to optimize the algorithm to maximize its advantages.

Author Contributions: J.H., conceptualization, methodology, software, data analysis, writing—original draft; Y.Z., supervision, investigation, writing—original draft; S.W., revision of the original manuscript; C.Z., conceptualization, supervision, funding acquisition. All authors have read and agreed to the published version of the manuscript.

Funding: This work is supported by the National Undergraduate Training Program on Innovation and Entrepreneurship (Number: 202510345033), the National Natural Science Foundation of China (Nos. 62272418,62102058), Basic public welfare research program of Zhejiang Province (No. LGG18E050011), and the Major Open Project of Key Laboratory for Advanced Design and Intelligent Computing of the Ministry of Education under grant ADIC2023ZD001.

Data Availability Statement: Data will be made available upon request.

Conflicts of Interest: The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. Ye, J.; Hao, L.; Li, X.; Cheng, H. A novel global path planning method for robot based on dual-source light continuous reflection. *ISA Trans.* **2024**, *150*, 15–29. [CrossRef]
2. Al Kamil, S.J.; Szabolcsi, R. Optimizing path planning in mobile robot systems using motion capture technology. *Results Eng.* **2024**, *22*, 102043. [CrossRef]
3. Liu, Y.; Wang, B.; Gao, X.; Fan, J.; Li, Q.; Dai, W. A passage time–cost optimal A* algorithm for cross-country path planning. *Int. J. Appl. Earth Obs. Geoinf.* **2024**, *130*, 103907. [CrossRef]
4. Wang, C.; Zang, X.; Song, C.; Liu, Z.; Zhao, J.; Ang, M.H., Jr. Virtual tactile POMDP-based path planning for object localization and grasping. *Measurement* **2024**, *230*, 114480. [CrossRef]
5. Liang, W.; Lou, M.; Chen, Z.; Qin, H.; Zhang, C.; Cui, C.; Wang, Y. An enhanced ant colony optimization algorithm for global path planning of deep-sea mining vehicles. *Ocean. Eng.* **2024**, *301*, 117415. [CrossRef]
6. Huang, X.; Liu, Y.; Huang, L.; Stikabakke, S.; Onstein, E. BIM-supported drone path planning for building exterior surface inspection. *Comput. Ind.* **2023**, *153*, 104019. [CrossRef]
7. Barnawi, A.; Kumar, K.; Kumar, N.; Thakur, N.; Alzahrani, B.; Almansour, A. Unmanned Ariel Vehicle (UAV) Path Planning for Area Segmentation in Intelligent Landmine Detection Systems. *Sensors* **2023**, *23*, 7264. [CrossRef]
8. Zhao, N.; Lu, W.; Sheng, M.; Chen, Y.; Tang, J.; Yu, F.; Wong, K. UAV-Assisted Emergency Networks in Disasters. *IEEE Wirel. Commun.* **2019**, *26*, 45–51. [CrossRef]
9. Zhou, Y.; Yang, J.; Guo, Z.; Shen, Y.; Yu, K.; Lin, J. An indoor blind area-oriented autonomous robotic path planning approach using deep reinforcement learning. *Expert Syst. Appl.* **2024**, *254*, 124277. [CrossRef]
10. Tang, Q.; Ma, L.; Zhao, D.; Sun, Y.; Wang, Q. A dual-robot cooperative arc welding path planning algorithm based on multi-objective cross-entropy optimization. *Robot. Comput. Integr. Manuf.* **2024**, *89*, 102760. [CrossRef]
11. Huo, F.; Zhu, S.; Dong, H.; Ren, W. A new approach to smooth path planning of Ackerman mobile robot based on improved ACO algorithm and B-spline curve. *Robot. Auton. Syst.* **2024**, *175*, 104655. [CrossRef]
12. Tao, B.; Kim, J.H. Mobile robot path planning based on bi-population particle swarm optimization with random perturbation strategy. *J. King Saud Univ. Comput. Inf. Sci.* **2024**, *36*, 101974. [CrossRef]
13. Cui, J.; Wu, L.; Huang, X.; Xu, D.; Liu, C.; Xiao, W. Multi-strategy adaptable ant colony optimization algorithm and its application in robot path planning. *Knowl. Based Syst.* **2024**, *288*, 111459. [CrossRef]
14. Li, M.; Li, B.; Qi, Z.; Li, J.; Wu, J. Optimized APF-ACO Algorithm for Ship Collision Avoidance and Path Planning. *J. Mar. Sci. Eng.* **2023**, *11*, 1177. [CrossRef]
15. Akay, R.; Yildirim, M.Y. Multi-strategy and self-adaptive differential sine–cosine algorithm for multi-robot path planning. *Expert Syst. Appl.* **2023**, *232*, 120849. [CrossRef]
16. Yildirim, M.Y.; Akay, R. An efficient grid-based path planning approach using improved artificial bee colony algorithm. *Knowl.-Based Syst.* **2025**, *318*, 113528. [CrossRef]
17. Tian, S.; Li, Y.; Kang, Y.; Xia, J. Multi-robot path planning in wireless sensor networks based on jump mechanism PSO and safety gap obstacle avoidance. *Future Gener. Comput. Syst.* **2021**, *118*, 37–47. [CrossRef]
18. Mac, T.T.; Copot, C.; Tran, D.T.; Keyser, R. A hierarchical global path planning approach for mobile robots based on multi-objective particle swarm optimization. *Appl. Soft Comput.* **2017**, *59*, 68–76. [CrossRef]
19. Das, P.K.; Jena, P.K. Multi-robot path planning using improved particle swarm optimization algorithm through novel evolutionary operators. *Appl. Soft Comput. J.* **2020**, *92*, 106312. [CrossRef]

20. Bandita, S.; Kumar, D.P.; Raghvendra, K. A modified cuckoo search algorithm implemented with SCA and PSO for multi-robot cooperation and path planning. *Cogn. Syst. Res.* **2023**, *79*, 24–42. [CrossRef]
21. Bandita, S.; Pradipta, K.D.; Manas Ranjan, K. Multi-robot cooperation and path planning for stick transporting using improved Q-learning and democratic robotics PSO. *J. Comput. Sci.* **2022**, *60*, 101637.
22. Kennedy, J.; Eberhart, R. Particle swarm optimization. In Proceedings of the ICNN'95—International Conference on Neural Networks, Perth, WA, Australia, 27 November–1 December 1995; Volume 4, pp. 1942–1948.
23. Mirjalili, S.; Mirjalili, S.M.; Lewis, A. Grey Wolf Optimizer. *Adv. Eng. Softw.* **2014**, *69*, 46–61. [CrossRef]
24. Mirjalili, S.; Lewis, A. The Whale Optimization Algorithm. *Adv. Eng. Softw.* **2016**, *95*, 51–67. [CrossRef]
25. Xue, J.; Shen, B. A novel swarm intelligence optimization approach: Sparrow search algorithm. *Syst. Sci. Control. Eng.* **2020**, *8*, 22–34. [CrossRef]
26. Zhu, D.; Wang, S.; Zhou, C.; Yan, S.; Xue, J. Human memory optimization algorithm: A memory-inspired optimizer for global optimization problems. *Expert Syst. Appl.* **2024**, *237*, 121597. [CrossRef]
27. Yang, X.S. Firefly Algorithms for Multimodal Optimization. In Proceedings of the Stochastic Algorithms: Foundations and Applications: 5th International Symposium, Sapporo, Japan, 26–28 October 2009; pp. 169–178.
28. Yang, X.S.; Deb, S. Cuckoo search via Lévy flights. In Proceedings of the 2009 World Congress on Nature & Biologically Inspired Computing, Coimbatore, India, 9–11 December 2009; IEEE: Piscataway, NJ, USA, 2009; pp. 210–214.
29. Gong, L.; Hou, G.; Huang, C. A two-stage MPPT controller for PV system based on the improved artificial bee colony and simultaneous heat transfer search algorithm. *ISA Trans.* **2022**, *132*, 428–443. [CrossRef]
30. Wang, S.; Pan, S.; Yang, Q.; Lin, C. Simulation Research of Photovoltaic MPPT Based on Fruit Fly Optimization Algorithm. *Sichuan Electr. Power Technol.* **2015**, *6*, 1–4.
31. Mirza, A.F.; Ling, Q.; Javed, M.Y.; Mansoor, M. Novel MPPT techniques for photovoltaic systems under uniform irradiance and Partial shading. *Sol. Energy* **2019**, *184*, 628–648. [CrossRef]
32. Eltamaly, A.M.; Al-Saud, M.S.; Abokhalil, A.G. A novel scanning bat algorithm strategy for maximum power point tracker of partially shaded photovoltaic energy systems. *Ain Shams Eng. J.* **2020**, *11*, 1093–1103. [CrossRef]
33. Li, X. An optimizing method based on autonomous animats: Fish-swarm algorithm. *Syst. Eng.-Theory Pract.* **2002**, *22*, 32–38.
34. Li, W.; Zhang, W.; Liu, B.; Guo, Y. The Situation Assessment of UAVs Based on an Improved Whale Optimization Bayesian Network Parameter-Learning Algorithm. *Drones* **2023**, *7*, 655. [CrossRef]
35. Yu, H.; Zhao, Z.; Heidari, A.A.; Ma, L.; Hamdi, M.; Mansour, R.F.; Chen, H. An accelerated sine mapping whale optimizer for feature selection. *iScience* **2023**, *26*, 107896. [CrossRef]
36. Donglin, Z.; Siwei, W.; Changjun, Z.; Yan, S. Manta ray foraging optimization based on mechanics game and progressive learning for multiple optimization problems. *Appl. Soft Comput. J.* **2023**, *145*, 110561.
37. Wu, L.; You, X.; Liu, S. Multi-ant colony algorithm based on cooperative game and dynamic path tracking. *Comput. Netw.* **2023**, *237*, 110077. [CrossRef]
38. Wang, J.; Li, S. An Improved Grey Wolf Optimizer Based on Differential Evolution and Elimination Mechanism. *Sci. Rep.* **2019**, *9*, 7181. [CrossRef]
39. Luo, K. Enhanced grey wolf optimizer with a model for dynamically estimating the location of the prey. *Appl. Soft Comput. J.* **2019**, *77*, 225–235. [CrossRef]
40. Dhargupta, S.; Ghosh, M.; Mirjalili, S.; Sarkar, R. Selective Opposition based Grey Wolf Optimization. *Expert Syst. Appl.* **2020**, *151*, 113389. [CrossRef]
41. Khatib, O. Real-Time Obstacle Avoidance for Manipulators and Mobile Robots. *Int. J. Robot. Res.* **1986**, *5*, 90–98. [CrossRef]
42. Dijkstra, E.W. A note on two problems in connexion with graphs. *Numer. Math.* **1959**, *1*, 269–271. [CrossRef]
43. Peter, H.; Nils, N.; Bertram, R. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Trans. Syst. Sci. Cybern.* **1968**, *4*, 100–107. [CrossRef]
44. Huang, T.; Fan, K.; Sun, W. Density gradient-RRT: An improved rapidly exploring random tree algorithm for UAV path planning. *Expert Syst. Appl.* **2024**, *252*, 124121. [CrossRef]
45. Zhou, Y.; Gross, L.; Codd, A. Inversion of 2D Magnetotelluric (MT) Data with Axial Anisotropy using Adaptive Particle Swarm Optimization (PSO). *J. Appl. Geophys.* **2024**, *226*, 105401. [CrossRef]
46. Chiheb, B.R.; Hichem, H.; Fethi, F.; Marai, A.; Zaafouri, A.; Chaari, A. Real-time implementation of a novel MPPT control based on the improved PSO algorithm using an adaptive factor selection strategy for photovoltaic systems. *ISA Trans.* **2023**, *146*, 496–510. [CrossRef] [PubMed]
47. Liu, H.; Zhang, X.; Tu, L. A Modified Particle Swarm Optimization Using Adaptive Strategy. *Expert Syst. Appl.* **2020**, *152*, 113353. [CrossRef]
48. Xing, Z.; Zhang, Z.; Shi, R.; Guo, Q.; Zeng, C. Filament-necking localization method via combining improved PSO with rotated rectangle algorithm for safflower-picking robots. *Comput. Electron. Agric.* **2023**, *215*, 108464. [CrossRef]
49. Liu, Z.; Tatsushi, N. Strategy dynamics particle swarm optimizer. *Inf. Sci.* **2022**, *582*, 665–703. [CrossRef]

50. Tareq, M.S.; Seyedali, M.; Yasser, A.E.; Khadija, D.; Saadat, I.; Laith, A. Velocity pausing particle swarm optimization: A novel variant for global optimization. *Neural Comput. Appl.* **2023**, *35*, 9193–9223. [CrossRef]
51. Hu, Q.; Zhou, N.; Chen, H.; Weng, S. Bayesian damage identification of an unsymmetrical frame structure with an improved PSO algorithm. *Structures* **2023**, *57*, 105119. [CrossRef]
52. Lin, S.; Liu, A.; Wang, J.; Kong, X. An improved fault-tolerant cultural-PSO with probability for multi-AGV path planning. *Expert Syst. Appl.* **2024**, *237*, 121510. [CrossRef]
53. Hsu, H.; Wang, C.; Nguyen, T.T.T.; Dang, T.; Pan, Y. Hybridizing WOA with PSO for coordinating material handling equipment in an automated container terminal considering energy consumption. *Adv. Eng. Inform.* **2024**, *60*, 102410. [CrossRef]
54. Gong, C.; Zhou, N.; Xia, S.; Huang, S. Quantum particle swarm optimization algorithm based on diversity migration strategy. *Future Gener. Comput. Syst.* **2024**, *157*, 445–458. [CrossRef]